

Experiment 5

Implementation of Modular Arithmetic Algorithms for Cryptographic Computations

Aim

To implement **Euclid's Algorithm, Extended Euclidean Algorithm, Fermat's Little Theorem, Euler's Theorem, and the Chinese Remainder Theorem (CRT)** using Python and demonstrate their applications in modular arithmetic for cryptographic computations.

Part A – Euclid's Algorithm

1. Start.
2. Read two integers a and b .
3. While $b \neq 0$:
 - Compute remainder = $a \bmod b$.
 - Set $a = b$.
 - Set $b = \text{remainder}$.
4. Repeat until $b = 0$.
5. Display a as the **Greatest Common Divisor (GCD)**.

Part B – Extended Euclidean Algorithm

1. Read two integers a and b .
2. Apply the Extended Euclidean Algorithm recursively.
3. If $b = 0$, return:

$$\text{gcd} = a, x = 1, y = 0$$

4. Otherwise, recursively calculate:

$$\text{gcd}, x_1, y_1$$

and compute:

$$\begin{aligned} x &= y_1 \\ y &= x_1 - \left\lfloor \frac{a}{b} \right\rfloor y_1 \end{aligned}$$

5. Display the GCD, x , and y .
6. If GCD = 1, calculate the modular inverse:

$$\boxed{a^{-1} = x \bmod b}$$

7. Display the modular inverse.

Part C – Fermat's Little Theorem

1. Read integer a and prime number p .
2. Check whether:

$$\text{gcd}(a, p) = 1$$

3. If they are not coprime, display an appropriate message.
4. Otherwise, compute:

$$\boxed{a^{p-1} \bmod p}$$

5. Display the result.

Part D – Euler's Theorem

1. Read integers a and n .
2. Calculate Euler's Totient Function $\phi(n)$ by counting the integers from 1 to n that are coprime with n .
3. Display $\phi(n)$.
4. Check whether:

$$\gcd(a, n) = 1$$

5. If they are not coprime, display an appropriate message.
6. Otherwise, compute:

$$\boxed{a^{\phi(n)} \bmod n}$$

7. Display the result.

Part E – Chinese Remainder Theorem

1. Read the number of congruence equations.
2. Read the **remainders** a_i and corresponding **moduli** m_i .
3. Calculate the product of all moduli:

$$M = m_1 m_2 \cdots m_n$$

4. For each modulus m_i , calculate the partial product:

$$M_i = \frac{M}{m_i}$$

5. Find the modular inverse of M_i modulo m_i using the **Extended Euclidean Algorithm**:

$$M_i^{-1} \pmod{m_i}$$

6. Calculate:

$$X = \sum a_i M_i M_i^{-1}$$

7. Find the final solution:

$$\boxed{x = X \bmod M}$$

8. Display the solution x .
9. Stop

Experiment No. 6: Implementation and Performance Comparison of Primality Testing Algorithms (Trial Division, Fermat Test, and Miller–Rabin Test)

Aim

To implement **Trial Division, Fermat Primality Test, and Miller–Rabin Primality Test** using Python and compare their computational efficiency and accuracy in determining whether a given number is prime.

Algorithm

Part A – Trial Division

1. Start.
2. Read an integer **n**.
3. Record the **start time**.
4. If $n \leq 1$, declare the number **Composite**.
5. Otherwise, check divisibility of n by integers from **2** to $\sqrt{n}$.
6. If any integer divides n exactly, declare the number **Composite**.
7. Otherwise, declare the number **Prime**.
8. Record the **end time**.
9. Calculate and display the **execution time**.

Part B – Fermat Primality Test

1. Read an integer **n**.
2. Record the **start time**.
3. If $n \leq 1$, declare **Composite**.
4. If $n = 2$ or $n = 3$, declare **Prime**.
5. If n is even, declare **Composite**.
6. Otherwise, choose a random integer a such that:

$$2 \leq a \leq n - 2$$

7. Compute:

$$a^{n-1} \bmod n$$

8. If the result is not 1, declare **Composite**.
9. Otherwise, declare **Probably Prime**.
10. Record the **end time**.
11. Calculate and display the **execution time**.

Part C – Miller–Rabin Primality Test

1. Read an integer **n**.
2. Record the **start time**.
3. If $n \leq 1$, declare **Composite**.
4. If $n = 2$ or $n = 3$, declare **Prime**.
5. If n is even, declare **Composite**.
6. Express:

$$n - 1 = 2^r \times d$$

where d is odd.

7. Choose a random integer a such that:

$$2 \leq a \leq n - 2$$

8. Compute:

$$x = a^d \bmod n$$

9. If $x = 1$ or $x = n - 1$, continue to the next iteration.

10. Otherwise, repeatedly square x :

$$x = x^2 \bmod n$$

11. If $x = n - 1$, continue to the next iteration.

12. If $n - 1$ is never obtained, declare **Composite**.

13. Repeat the test for **5 iterations**.

14. If all iterations are passed, declare **Probably Prime**.

15. Record the **end time**.

16. Calculate and display the **execution time**.

17. Compare the execution times of **Trial Division, Fermat Test, and Miller–Rabin Test**.

18. Stop.

Experiment No. 7: Implementation of RSA Key Generation, Encryption, and Decryption for Secure Communication

Aim

To implement **RSA key generation, encryption, and decryption using Python** by generating a **2048-bit RSA key pair**, encrypting the plaintext using the **public key**, and decrypting the ciphertext using the **private key** for secure communication.

Algorithm

1. Start.
2. Import the required Python libraries for RSA key generation and encryption.
3. Generate a 2048-bit RSA key pair.
4. Extract the public key and private key from the generated RSA key pair.
5. Display the public key and private key.
6. Read the plaintext message from the user.
7. Convert the plaintext message into bytes.
8. Create an RSA cipher using the public key.
9. Encrypt the plaintext using the public key with OAEP padding.
10. Convert and display the ciphertext in hexadecimal format.
11. Create an RSA cipher using the private key.
12. Decrypt the ciphertext using the private key.
13. Convert the decrypted data from bytes back into text.
14. Display the decrypted plaintext message.
15. Stop.